

Fördjupning i Polars

Fördjupning i Polars. Avancerad python, data scientist.

Syfte

Jag ville lära mig grunderna i Polars och samtidigt förstå varför det ofta beskrivs som snabbare än Pandas. Under utbildningen har vi mest jobbat med Pandas, men Polars har dykt upp gång på gång i artiklar och diskussioner. Istället för att bara ta det för givet ville jag testa själv och se vad skillnaden faktiskt innebär – både i hur man skriver koden och i hur snabbt den kör.

Mer specifikt ville jag:

- lära mig läsa in, filtrera, sortera och gruppera data i Polars
- förstå vad som ligger bakom prestandaskillnaden mot Pandas rent tekniskt
- göra en egen jämförelse på en större datamängd

Området och dess relevant

Polars är ett DataFrame-bibliotek skrivet i Rust med ett Python-API ovanpå. Det har blivit allt populärare inom data science, mycket tack vare att det kan hantera stora datamängder effektivt.

Relevansen för Data Science-yrkesrollen tycker jag är ganska tydlig. Under utbildningen har vi jobbat med ganska små dataset där Pandas fungerar hur bra som helst. Men i verkliga projekt – loggdata, sensordata och mätvärden från elnätet – kan datamängderna bli betydligt större, och då spelar valet av bibliotek roll.

Att kunna både Pandas och Polars, och veta när det ena kan vara ett bättre val än det andra, känns som praktisk kunskap att ta med sig.

Viktiga begrepp

Kolumnbaserad lagring (Apache Arrow)

Polars använder en kolumnbaserad representation av data och bygger på Apache Arrow-formatet. Det gör att data kan bearbetas effektivt, särskilt när man arbetar med hela kolumner.

Multi-threading

Polars kan använda flera CPU-kärnor samtidigt. Det betyder att flera delar av arbetet kan köras parallellt utan att jag behöver skriva extra kod för det.

Lazy evaluation

Med lazy evaluation kan jag bygga upp flera operationer först och låta Polars optimera planen innan resultatet räknas fram. Då används till exempel `scan_csv()` istället för `read_csv()` och `collect()` används på slutet. Det påminner lite om hur en SQL-databas planerar en fråga.

Striktare typning

Polars är mer noggrant med datatyper än Pandas. Det kan göra att man behöver vara mer tydlig med typer, men det kan också minska risken för oväntade resultat.

Genomförande

Data

Jag byggde ett syntetiskt dataset med daglig elförbrukning för fem svenska städer: Stockholm, Göteborg, Malmö, Uppsala och Umeå. Datan är uppdelad på hushåll, industri och kommersiell verksamhet och täcker hela 2025.

Datasetet innehåller 5 475 rader. Jag byggde in säsongsvariation, där förbrukningen är högre under vintern. Jag lade också in ett antal saknade värden i temperaturkolumnen för att kunna öva på att hantera sådana värden.

Verktyg och arbetsgång

Jag jobbade i en Jupyter-notebook och använde:

- polars för datahantering
- pandas för jämförelsen
- numpy för att generera testdata
- time för att mäta körtider

Arbetsgången var ungefär:

1. Läste in datan med `pl.read_csv()` och tittade på schema, antal rader och saknade värden.
2. Filtrerade fram delmängder, till exempel dagar i Stockholm med hög förbrukning.
3. Sorterade på en och flera kolumner.
4. Lade till nya kolumner med `with_columns()`, bland annat kostnadsberäkning och temperaturkategori.
5. Grupperade och summerade per region, region och sektor samt per månad.
6. Testade lazy evaluation med `scan_csv()` och tittade på planen med `explain()`.
7. Gjorde en prestandajämförelse mot Pandas på en testfil med en miljon rader.

Ett typiskt exempel på gruppering i Polars:

```
resultat = (  
    df.group_by("region")  
  
    .agg(pl.col("forbrukning_kwh").sum().alias("total_forbrukning_kwh"))  
    .sort("total_forbrukning_kwh", descending=True)  
)
```

Motsvarande i Pandas:

```
resultat = (  
    df.groupby("region")
```

```
df.groupby("region")["forbrukning_kwh"]
.sum()
.reset_index(name="total_forbrukning_kwh")
.sort_values("total_forbrukning_kwh", ascending=False)
)
```

Resultatet blir samma typ av sammanställning, men jag tycker Polars-versionen är ganska tydlig eftersom man ser kolumnen som summeras och namnet på resultat-kolumnen i samma uttryck.

Problem jag stötte på

Det största hindret var egentligen inte syntaxen, utan att vänja sig vid att tänka i uttryck med `pl.col()` istället för att referera till kolumner som vanliga variabler. I början var det lätt att blanda ihop själva uttrycket med resultatet av uttrycket. Efter att ha testat några exempel blev det mer naturligt.

Ett annat litet problem var att Polars är striktare med datatyper. Det kan kännas ovant när man kommer från Pandas, men jag tycker samtidigt att det gör koden tydligare.

Resultat

Notebooken täcker in det jag ville testa: inläsning, filtrering, sortering, beräkningar, gruppering och lazy evaluation. Koden körs utan problem.

Grupperingen per region gav till exempel:

Region	Total förbrukning kWh	Snitt förbrukning kWh
Stockholm	470 684,5	429,8
Göteborg	401 241,0	366,4
Malmö	363 570,1	332,0
Uppsala	218 410,6	199,5
Umeå	182 051,4	166,3

När jag grupperade per månad syntes säsongsvariationen tydligt. Förbrukningen var högre i januari, februari och december än under sommarmånaderna, vilket var väntat eftersom jag byggde in den variationen i datan.

Prestanda jämförelse

För jämförelsen genererade jag en separat testfil med en miljon rader slumpad numerisk data. Jag mätte tiden för att läsa in filen, filtrera data och sedan gruppera och beräkna medelvärden. Samma operationer kördes i Pandas och Polars, både eager och lazy.

Metod	Tid	Jämfört med Pandas
Pandas	1,218 s	–
Polars (eager)	0,522 s	cirka 2,3 gånger snabbare
Polars (lazy)	0,460 s	cirka 2,6 gånger snabbare

I mitt test var Polars alltså mer än dubbelt så snabbt som Pandas för den här kedjan av operationer. Lazy-läget var också något snabbare i testet.

Siffrorna ska dock inte ses som ett exakt benchmark. Jag körde varje variant en gång och testet gjordes på min dator.

Begränsningar och möjliga förbättringar

Jag har medvetet hållit mig till grundläggande funktioner i Polars. Därför har jag inte testat allt som biblioteket erbjuder.

Några saker som skulle kunna utvecklas:

- streaming av data som inte får plats i minnet
- egna plugins eller UDF:er
- integration mot databaser och molntjänster
- arbete med Parquet och andra dataformat

Datan jag använde är syntetisk. Det var praktiskt för uppgiften eftersom jag kunde bygga in säsongvariation och saknade värden, men den representerar inte verklig elförbrukning.

Prestandetestet är också begränsat eftersom varje variant bara kördes en gång. Ett bättre test skulle använda flera körningar och jämföra median eller genomsnitt.

Om jag hade haft mer tid hade jag velat testa mot en riktig, öppen datamängd, till exempel elförbrukningsdata från Energimyndigheten, och även testa data som är större än minnet.

Koppling till yrkesrollen

Polars är relevant inom Data Science eftersom data kan bli stora och ofta behöver filtreras, grupperas och aggregeras innan den används i analys eller maskininlärning.

Under utbildningen har vi mest jobbat med mindre dataset där Pandas fungerar bra. I verkliga projekt kan datamängderna däremot bli större. Därför tycker jag att det är bra att förstå både Pandas och Polars och kunna välja verktyg beroende på situation.

För mig är kopplingen till eldata extra intressant. Mätvärden från elnätet kan snabbt bli stora eftersom mätningar samlas in över tid från många olika platser. Polars funktioner för filtrering, gruppering och aggregering passar därför bra för ett sådant exempel.

Källor:

- Polars officiella dokumentation: <https://docs.pola.rs/>
- Polars GitHub-repository: <https://github.com/pola-rs/polars>
- Pandas officiella dokumentation: <https://pandas.pydata.org/docs/>

Se även Jupyter-notebooken `polars_fordjupning.ipynb` för fullständig körd kod med förklaringar.

Självreflektion

Vad lärde du dig som du inte kunde innan?

Jag hade inte jobbat med Polars innan uppgiften. Nu känner jag mig mer bekväm med grunderna, till exempel inläsning, filtrering, sortering, `with_columns()`, `group_by()` och `agg()`. Jag har också fått en bättre förståelse för varför Polars kan vara snabbt.

Vad var svårast att förstå eller genomföra?

Lazy evaluation var det som tog längst tid att förstå. Syntaxen är ganska enkel, men det tog tid att förstå att Polars först kan bygga en plan och sedan optimera den innan resultatet räknas fram. Att använda `explain()` hjälpte mig att förstå detta bättre.

Vilket tekniskt val är du mest nöjd med och varför?

Jag är nöjd med att jag byggde ett eget dataset istället för att bara använda ett färdigt dataset. Då kunde jag själv lägga in säsongsvariation och saknade värden och sedan kontrollera resultaten.

Vad hade du gjort annorlunda om du började om?

Jag hade lagt mer tid på prestandatestet. Jag hade kört varje test flera gånger och använt ett mer stabilt sätt att jämföra tiderna. Jag hade också gärna testat en riktig datamängd.

Vad skulle vara ett naturligt nästa steg?

Ett naturligt nästa steg är att testa Polars med en riktig öppen datamängd och undersöka streaming för större filer.

Vilket betyg tycker du själv att arbetet motsvarar?

Jag tycker att arbetet motsvarar G. Jag har valt ett relevant område, avgränsat uppgiften, använt officiell dokumentation, byggt en fungerande lösning och kan förklara de viktigaste delarna. Prestandatestet kan förbättras, men det ger ändå en tydlig bild av skillnaden i just mitt test.